Université du Québec à Chicoutimi

Département des sciences appliquées

Laboratoire 3

Interface graphique du système de gestion de tickets

Rapport

Par

Équipe : Zongo T Julien, Zongo A Razack

2026-10-26

Dépôt : github.com/zongo-julien/Laboratoire3_6GEI311_Architecture_des_logiciels

Table des matières

1. Introduction 4
2. Le modèle avant l'interface 4
3. Architecture de l'interface 6
    3.1 Modèle, vues, contrôleurs 6
    3.2 Un contrat figé dès le départ 7
    3.3 Le patron Observateur 7
    3.4 Les refus du modèle 8
    3.5 Données, images et sessions 9
    3.6 Diagramme de classes 9
4. Les fenêtres planifiées et implémentées 10
    4.1 F1 Connexion 10
    4.2 F2 Liste Admin (vue Admin) 12
    4.3 F7 Confirmation du lot 12
    4.4 F8 Rapport d'exécution 14
    4.5 F3 Mes tickets 16
    4.6 F4 Nouveau ticket 20
    4.7 F5 Éditeur de description 20
    4.8 F6 Détail d'un ticket 23
5. Tactiques d'utilisabilité 26
6. Tactiques de modificabilité 27
7. Tests et validation 28
    7.1 Tests automatisés 28
8. Organisation du travail 29

9. Difficultés et ce que nous avons appris 30
10. Conclusion 31
Annexe A. Lancer l'application et les tests 32
Annexe B. Structure du code de l'interface 32

1. Introduction

Au Laboratoire 2, notre système de tickets ne se pilotait que par main.py, scénario après scénario. Au Laboratoire 3, il a une interface graphique. Un Admin et plusieurs utilisateurs ouvrent chacun leur fenêtre depuis la même connexion. Les utilisateurs créent et décrivent leurs tickets, l'Admin les fait avancer dans leur cycle de vie, et chaque fenêtre ouverte voit les changements des autres sans qu'on ait à la recharger.

Nous avons gardé les contraintes du dépôt : Python 3.10 ou plus récent, tkinter, aucune dépendance externe. Les images jointes sont limitées au PNG et au GIF, les deux formats que tkinter affiche sans Pillow.

Le rapport suit l'ordre dans lequel nous avons travaillé. La section 2 résume ce que nous avons changé dans le modèle avant d'écrire la première fenêtre, et la section 3 l'architecture de l'interface. La section 4 présente les huit fenêtres et leurs boîtes de dialogue, chaque fois avec la maquette Figma puis la capture de la fenêtre réelle. Les sections suivantes portent sur les tactiques d'utilisabilité, les tests et l'organisation de l'équipe.

2. Le modèle avant l'interface

Nous avons relu ensemble le modèle du Lab 2 avec une seule question en tête : qu'est-ce qu'une interface va lui demander ? Un script comme main.py ne se plaint jamais d'une fenêtre qui ne se rafraîchit pas. La revue a donc fait ressortir des manques que nos scénarios ne pouvaient pas voir. Le tableau 1 donne chaque constat et la décision qui en est sortie.

Tableau 1. Constats de la revue et décisions.

Constat de la revue	Décision	Tâche
Un utilisateur voit les tickets qu'il a créés, mais ne voyait pas ceux qui lui sont assignés.	SystemeTickets.ticketsAssignesA(user).	M1
Pour griser un bouton, la vue devrait recopier les règles de la machine à états.	transitionsPermises et transitionPossible(statut) interrogent l'état courant sans rien modifier. Un test vérifie qu'elles disent la même chose que gererTransition().	M2
Une vue ne peut lire ni le type ni le contenu d'un élément de description.	Propriétés en lecture seule typeArtefact et contenu.	M3
On ne peut décrire un ticket qu'en joignant un fichier.	DescriptionTexteSaisi, créée par FabriqueDescription.creerSaisie(). Saisi par soi même.	M4
Rien ne permet de renoncer à une édition avant de l'appliquer, on ne faisait que la pause.	SessionEdition.abandonner() et estOuverte.	M5
Un ticket est OUVERT, et visible de tous, avant même d'être décrit.	SessionCreation : le ticket n'entre dans le dépôt qu'à appliquer().	M6
Une fenêtre ouverte n'apprend pas qu'un ticket a changé.	Patron Observateur : ObservateurTickets, SujetTickets, relais par SystemeTickets.	M8
Deux sessions peuvent éditer le même ticket en même temps.	Une seule session ouverte par ticket : sessionEnCours(), sessionsOuvertes.	M9
Tout est perdu à la fermeture du programme.	Sauvegarde JSON : DepotJSON, FabriqueEtat, Ticket.restaurer(), discuter avec le professeur	M10

Juste avant l'interface, deux retouches de plus ont été nécessaires. SessionEdition.ticket indique le ticket visé par une session, ce dont le contrôleur d'édition a besoin. StockageImages copie chaque image jointe dans donnees/images/ sous un nom unique : sans cette copie, une image que l'utilisateur déplace ou supprime de son disque disparaîtrait de la description.

3. Architecture de l'interface

3.1 Modèle, vues, contrôleurs

L'interface ajoute deux paquets, ihm/vues et ihm/controleurs, au-dessus du modèle. Chaque couche a des droits précis, résumés au tableau 2.

Tableau 2. Les trois couches et leurs droits.

Couche	Paquets	Ce qu'elle a le droit de faire
Modèle	acteurs, modele, etats, description, sessions, commandes, persistance	Porter les règles métier. Il ne connaît pas tkinter.
Vues	ihm/vues	Lire le modèle par ses propriétés en lecture seule et l'observer. Une vue ne le modifie jamais.
Contrôleurs	ihm/controleurs	Faire toutes les modifications, en passant par Admin, User, les sessions ou LotAdmin. Jamais ticket.updateStatus(), setEtat() ni un attribut privé.

Une règle a beaucoup simplifié les tests : seul ControleurPrincipal crée des fenêtres, et les trois autres contrôleurs n'importent ni tkinter ni les vues. Grâce à cette séparation, ControleurAdmin, ControleurUser et ControleurEdition ysont testés sans ouvrir une seule fenêtre, avec un faux contrôleur principal. Un test lance même un sous-processus pour vérifier que l'import de ControleurEdition ne charge pas tkinter.

ControleurPrincipal est le seul à connaître tkinter côté contrôleurs. Il charge et sauvegarde les données, ouvre chaque fenêtre avec son contrôleur, et tient un registre des sessions d'édition (section 3.5).

3.2 Un contrat figé dès le départ

Dès la première étape du sprint (I1), avant d'écrire une seule vraie fenêtre, nous avons figé les constructeurs des vues et les signatures des contrôleurs, puis créé chaque fenêtre sous forme d'ébauche. L'application tournait ainsi de bout en bout, et chacun a pu remplir ses propres fichiers sans attendre l'autre. Le tableau 3 donne ce contrat. Pendant tout le sprint, nous n'y avons ajouté qu'une propriété, ControleurEdition.nombreImages, dont F5 a besoin pour griser « Créer le ticket ».

Tableau 3. Le contrat figé par l'étape I1.

Fenêtre	Vue et constructeur	Contrôleur
F1 Connexion	VueConnexion(parent, principal)	ControleurPrincipal
F2 Liste Admin	VueListeAdmin(parent, controleur)	ControleurAdmin(principal, admin)
F3 Mes tickets	VueListeUser(parent, controleur)	ControleurUser(principal, user)
F4 Nouveau ticket	VueNouveauTicket(parent, principal, user)	ControleurPrincipal
F5 Éditeur	VueEdition(parent, controleur)	ControleurEdition(principal, session)
F6 Détail	VueDetail(parent, systeme, tickets, index)	aucun (lecture seule)
F7 Confirmation du lot	VueLot(parent, controleur, action, tickets)	ControleurAdmin
F8 Rapport	VueRapport(parent, rapport, libelle)	aucun (lecture seule)

Toutes les fenêtres héritent de FenetreBase(tk.Toplevel, ObservateurTickets), dont le constructeur reçoit la liste des sujets à observer. Elle fournit rafraichir(), à redéfinir, fermer() et executer(), décrites plus bas.

3.3 Le patron Observateur

FenetreBase abonne la fenêtre à ses sujets à l'ouverture et la désabonne à la fermeture. Les listes F2 et F3 et le détail F6 sont abonnés au dépôt, SystemeTickets. Le dépôt observe lui-même chacun de ses tickets et retransmet leurs modifications : une fenêtre n'a donc qu'un

abonnement à gérer, au lieu d'un par ticket. Cela se voit facilement à l'écran : avec Ambroise Fleury (Admin) et T Julien ouverts côte à côte, le ticket que T Julien crée apparaît dans la liste d'Ambroise sans qu'il touche à rien.

L'ordre des opérations dans fermer() compte : se désabonner d'abord, détruire ensuite. Dans l'autre ordre, le dépôt notifierait une fenêtre détruite et tkinter lèverait TclError au premier rafraîchissement. C'est le compromis du patron vu en cours : l'observateur doit penser à partir, sinon il continue d'être appelé et reste en mémoire.

Les sessions, elles, ne sont pas des sujets observables. Pour qu'un brouillon mis en pause depuis F5 apparaisse aussitôt dans F3, la vue relit l'état des sessions de l'utilisateur toutes les 500 ms et ne se redessine que si cet état a changé. C'est une entorse au patron, et nous l'assumons : rendre les sessions observables aurait touché au modèle en fin de sprint, pour un résultat identique à l'écran.

3.4 Les refus du modèle

Une interface ne doit pas planter parce que le modèle refuse une opération. FenetreBase.executer(fonction, *args) appelle le contrôleur et attrape les cinq exceptions métier de modele/exceptions.py. Le message s'affiche dans une boîte « Opération refusée » (figure 1) et la fenêtre reste ouverte. Toute autre exception est un bug : elle remonte dans la console au lieu d'être cachée.

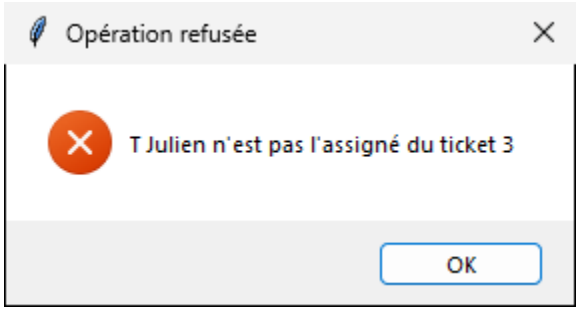


Figure 1. Refus métier affiché par executer() : un utilisateur qui n'est pas l'assigné tente de soumettre un ticket à validation.

En pratique, cette boîte apparaît rarement, et c'est voulu. Les boutons qui mèneraient à un refus sont grisés d'avance, en posant au modèle la même question qu'il se posera lui-même : transitionPossible(), peutSupprimer(), peutSoumettre(). L'exception reste un filet de sécurité. Un détail nous a surpris en chemin : executer() retourne None après un refus, mais aussi quand la fonction appelée ne retourne rien. Pour ne fermer F4 ou F5 qu'en cas de succès, nous enveloppons l'appel dans une petite fonction qui retourne True.

3.5 Données, images et sessions

Au démarrage, l'application charge donnees/sauvegarde.json s'il existe, sinon les données de démonstration donnees/demo.json. Une sauvegarde illisible est signalée, puis l'application démarre sur la démonstration. À la fermeture de la connexion, tout le système est sauvegardé dans donnees/sauvegarde.json, que Git ignore. Si cette sauvegarde échoue, l'application le dit et demande « Quitter sans sauvegarder ? », « Non » par défaut.

La démonstration est générée par outils/generer_demo.py avec l'API du modèle : sessions de création pour les tickets, méthodes d'Admin et de User pour leur cycle de vie. Elle contient les cinq acteurs de main.py et huit tickets qui couvrent les cinq statuts. Ses trois images PNG sont dessinées par le script lui-même, avec zlib et struct, et elle ne contient aucune vidéo. Un test vérifie qu'elle se recharge sans avertissement.

Chaque image ajoutée dans F5 est copiée par StockageImages. ControleurEdition garde une pile de copies alignée sur la pile de la session : le chemin de la copie pour une image, rien pour un commentaire. Annuler le dernier ajout supprime ainsi la bonne copie, et abandonner la session les supprime toutes. Un seul ControleurEdition existe par session, dans un registre de ControleurPrincipal. Il survit à la fermeture de F5, ce qui garde le texte tapé pendant une pause. Les sessions encore ouvertes ne sont pas sauvegardées, et la connexion l'annonce avant de quitter (section 4.1).

3.6 Diagramme de classes

Le diagramme de classes pour le Lab 3 (tâche U1) reprend celui de la Partie 2 du Lab 2 et y ajoute ce que le Lab 3 a introduit : le patron Observateur, la session de création, la persistance (DepotJSON, StockageImages) et le paquet ihm de l'interface. Le fichier source, documents/Lab3_6GEI311_diagramme_classes.drawio, compte cinq pages : le modèle, l'interface, les instanciations, les exceptions et le cycle de vie du ticket. Elles sont reproduites en paysage à l'annexe C, où elles restent lisibles.

4. Les fenêtres planifiées et implémentées

Le tableau 4 recense les huit fenêtres.

Tableau 4. Les huit fenêtres.

Fenêtre	Rôle	Vue	Responsable
F1 Connexion	Choisir l'acteur avec lequel ouvrir une session	vue_connexion.py	Julien
F2 Liste Admin	Tout le dépôt, filtre par statut, actions en lot	vue_liste_admin.py	Abdoul
F3 Mes tickets	Tickets créés, tickets assignés et brouillons	vue_liste_user.py	Abdoul
F4 Nouveau ticket	Titre et priorité d'un nouveau ticket	vue_nouveau_ticket.py	Julien
F5 Éditeur	Texte, images et commentaires, en création ou en édition	vue_edition.py	Julien
F6 Détail	Lecture d'un ticket, commune aux deux rôles	vue_detail.py	Abdoul
F7 Confirmation du lot	Opérations de l'Admin avant application	vue_lot.py	Abdoul
F8 Rapport	Succès et refus après application	vue_rapport.py	Abdoul

4.1 F1 Connexion

La connexion liste les acteurs de l'annuaire, nom à gauche et rôle à droite. « Ouvrir la session », un double-clic ou la touche Entrée ouvre F2 pour l'Admin et F3 pour un utilisateur. La connexion reste ouverte après coup : c'est elle qui permet d'avoir plusieurs acteurs à l'écran en même temps. Il n'y a pas de mot de passe, l'authentification sortant du cadre du laboratoire.

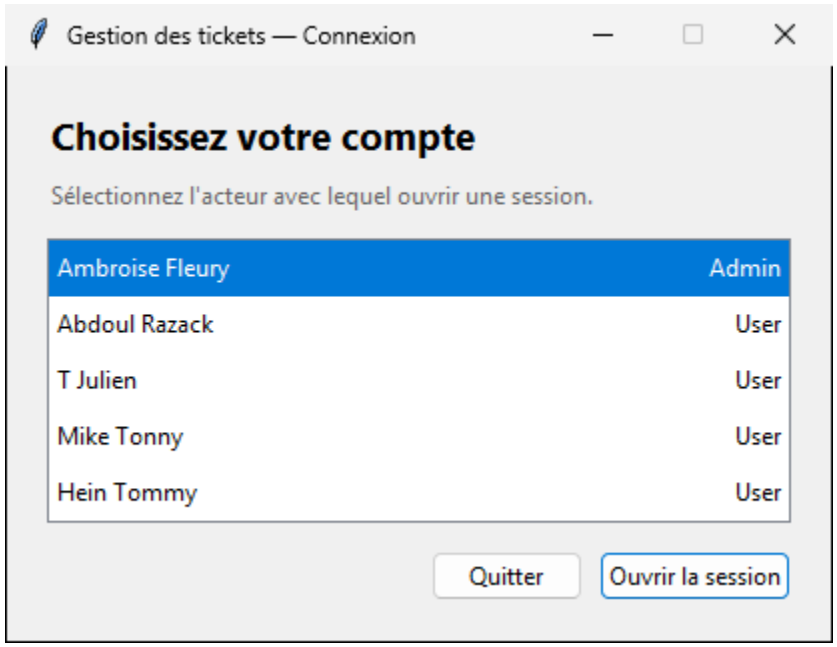


Figure 2. F1 Connexion : fenêtre réelle.

Fermer la connexion, c'est quitter l'application. S'il reste des sessions d'édition ou de création ouvertes, une confirmation l'annonce avec « Non » par défaut, puisque ces sessions ne sont pas sauvegardées.

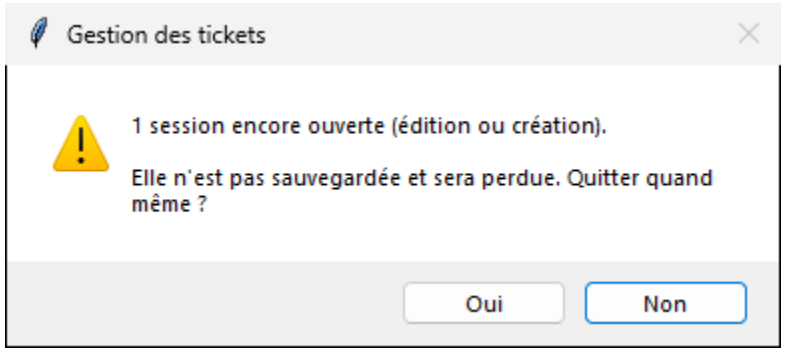


Figure 3. Confirmation en quittant : boîte réelle, avec une session de création en pause.

4.2 F2 Liste Admin (vue Admin)

F2 montre tout le dépôt : numéro, titre, priorité, statut, créateur et assigné, avec un filtre par statut. L'Admin sélectionne un ou plusieurs tickets (Ctrl+clic) puis choisit une action. Un bouton n'est actif que si au moins un ticket sélectionné accepte la transition. Sur la capture, deux tickets OUVERT sont sélectionnés : « Assigner… » et « Fermer » sont actifs, « Désassigner » et « Valider » sont grisés. Aucune action ne modifie un ticket directement, chacune ouvre F7.

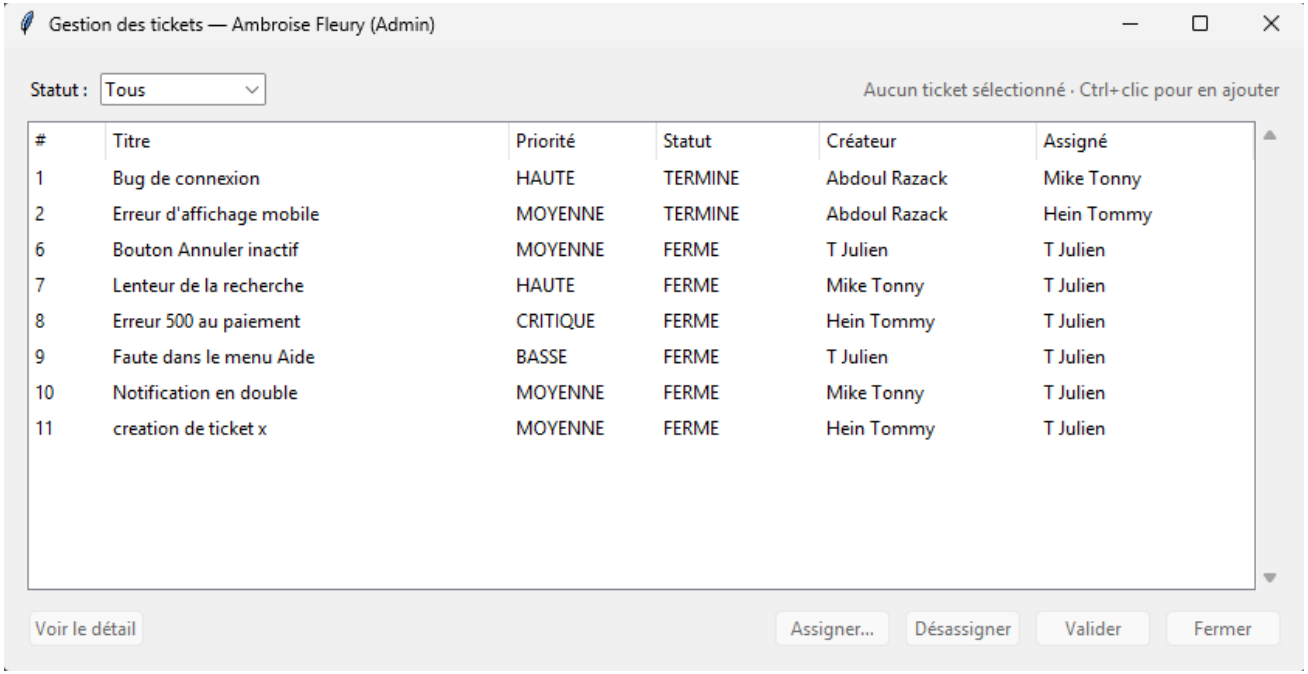


Figure 4. F2 Liste Admin : fenêtre réelle, deux tickets OUVERT sélectionnés.

4.3 F7 Confirmation du lot

F7 affiche exactement les opérations que le lot appliquera, telles que LotAdmin les décrit. Pour « Assigner… », l'Admin choisit d'abord l'utilisateur : aucun nom n'est proposé d'avance, et « Appliquer » reste grisé tant qu'il n'a pas choisi. Sur la capture, la sélection contient aussi le ticket #1, déjà TERMINE. Il reste dans le lot et sera refusé, avec son motif, dans le rapport.

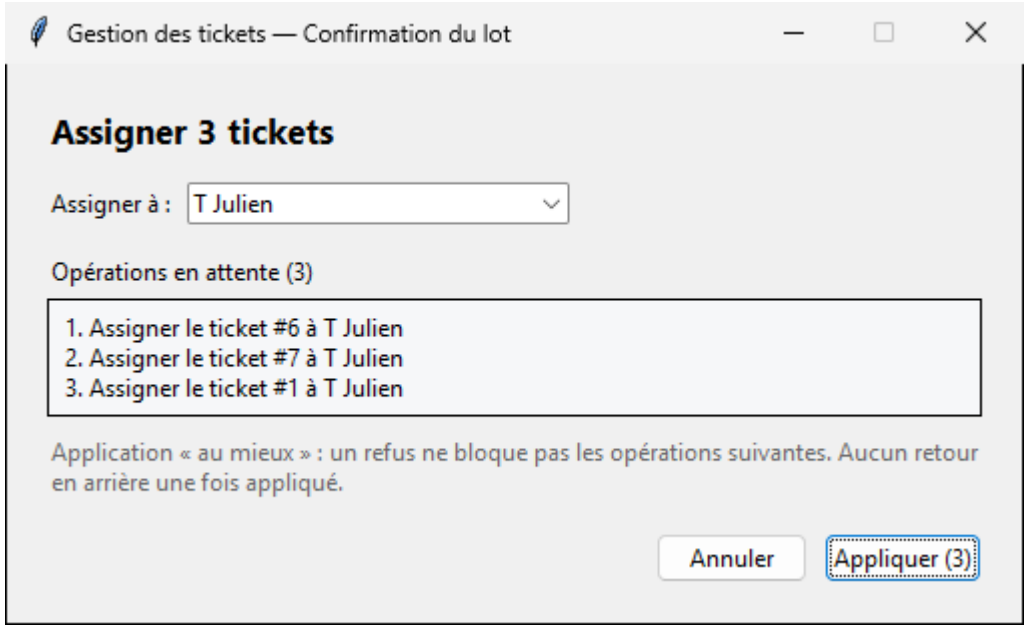


Figure 5. F7 pour « Assigner… » : trois opérations en attente.

Pour « Fermer », la fenêtre change de ton. Un bandeau rouge rappelle que l'action est définitive, le bouton devient « Fermer définitivement » et le focus est sur « Annuler » : la touche Entrée ne ferme donc aucun ticket par réflexe.

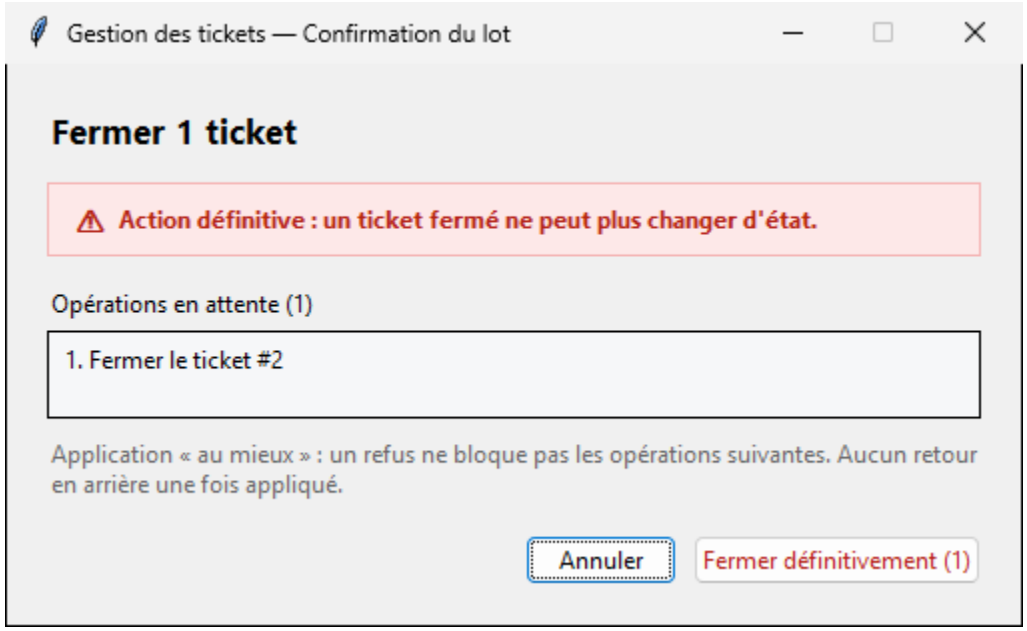


Figure 6. F7 pour « Fermer » : avertissement et focus sur « Annuler ».

4.4 F8 Rapport d'exécution

Après « Appliquer », F8 donne le bilan, puis chaque succès et chaque refus avec son motif. L'application se fait « au mieux » : le refus du ticket #1 n'a pas empêché l'assignation des tickets #6 et #7. La même fenêtre sert au rapport d'une session de création ou d'édition.

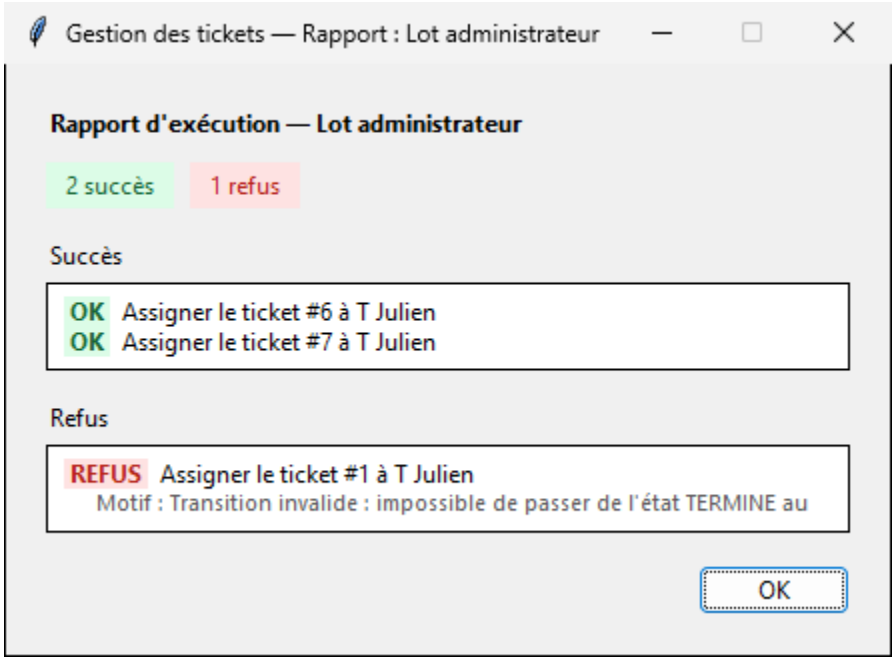


Figure 7. F8 après le lot d'assignation : deux succès, un refus motivé.

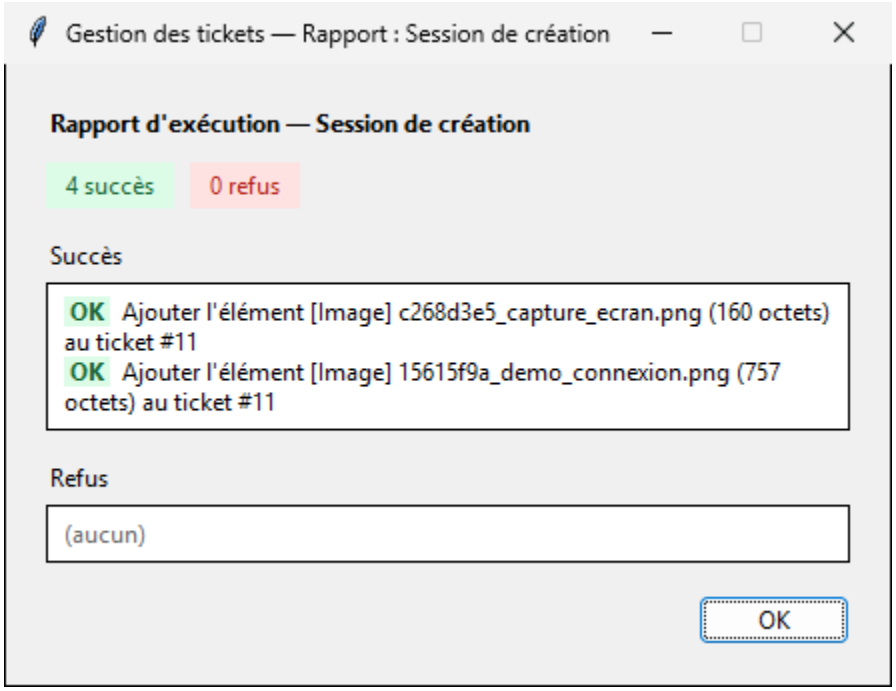


Figure 8. F8 après la création d'un ticket : quatre modifications appliquées (deux images, un commentaire et le texte tapé).

4.5 F3 Mes tickets

F3 ne montre que ce qui concerne l'utilisateur, en trois onglets : les tickets qu'il a créés, ceux qui lui sont assignés et ses brouillons. Un brouillon est une création mise en pause, dont le ticket n'est pas encore dans le dépôt. Le titre de chaque onglet donne son nombre de lignes, et la colonne « Édition » signale une session en cours ou en pause.

Les boutons suivent les règles du modèle. « Supprimer » n'est actif que pour un ticket créé par l'utilisateur et jamais assigné, « Soumettre à validation » que pour un ticket qui lui est assigné et dont l'état le permet, « Éditer » que pour ses propres tickets. Sur un brouillon, ou sur un ticket dont l'édition est en pause, le bouton devient « Reprendre l'édition ».

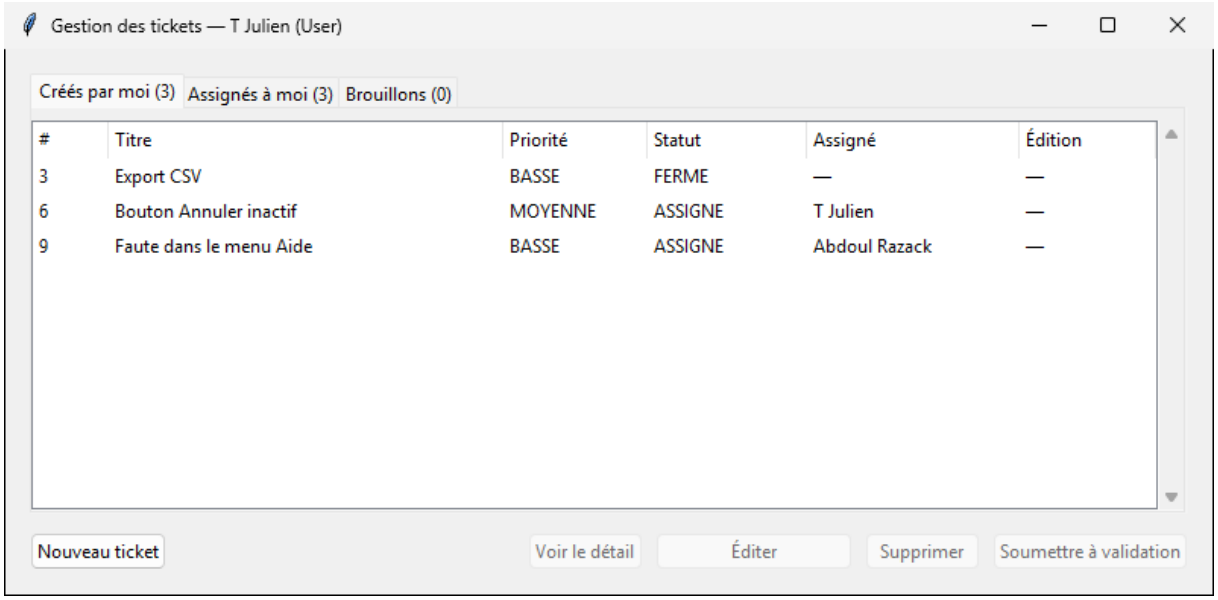


Figure 9. F3, onglet « Créés par moi ».

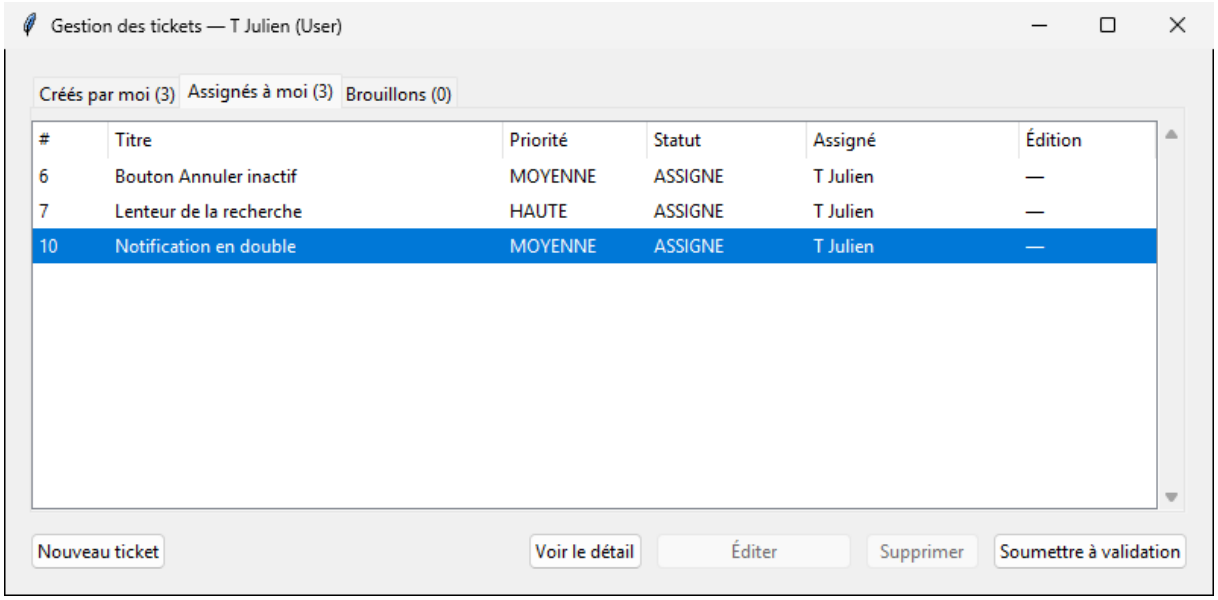


Figure 10. F3, onglet « Assignés à moi » : « Soumettre à validation » est actif.

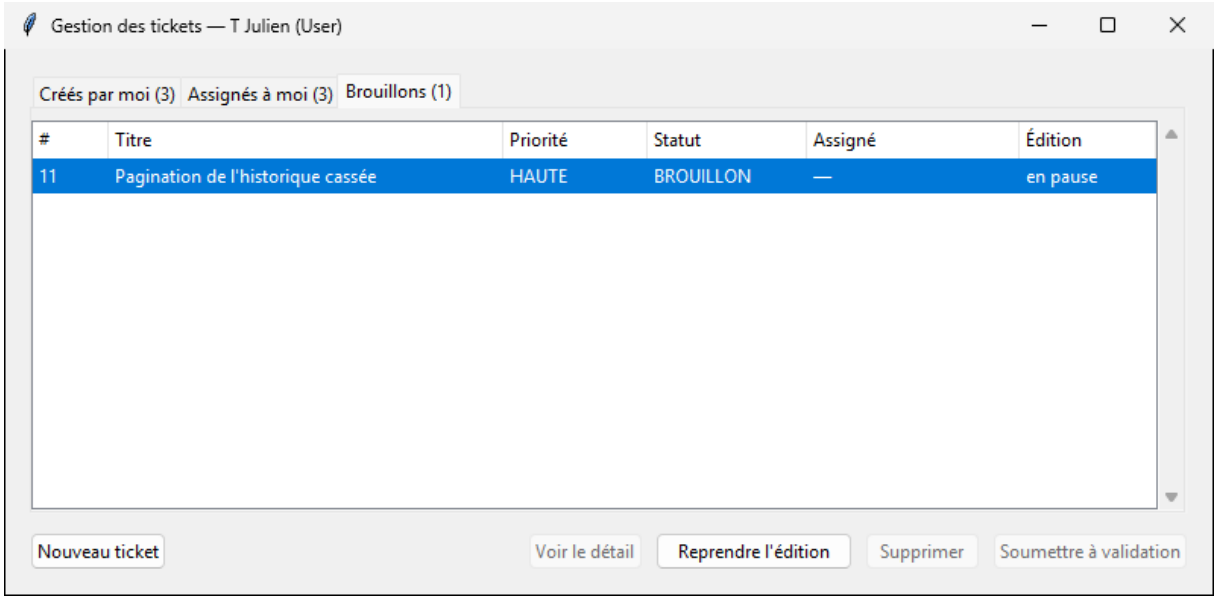


Figure 11. F3, onglet « Brouillons » : la création du ticket #11, en pause.

La suppression demande une confirmation, « Non » par défaut, car elle ne peut pas être annulée.

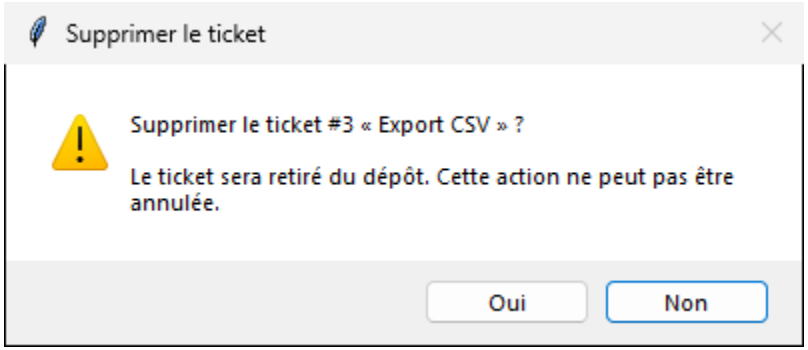


Figure 12. Confirmation de suppression d'un ticket depuis F3.

4.6 F4 Nouveau ticket

F4 ne demande que le titre et la priorité. « Continuer » reste grisé tant que le titre est vide. Il crée le ticket et sa SessionCreation sans toucher au dépôt, puis ouvre l'éditeur. « Annuler », Échap ou la croix ferment F4 sans aucun appel au modèle. Séparer F4 et F5 nous a ramenés à la maquette d'origine : le ticket n'existe pour les autres qu'une fois décrit.

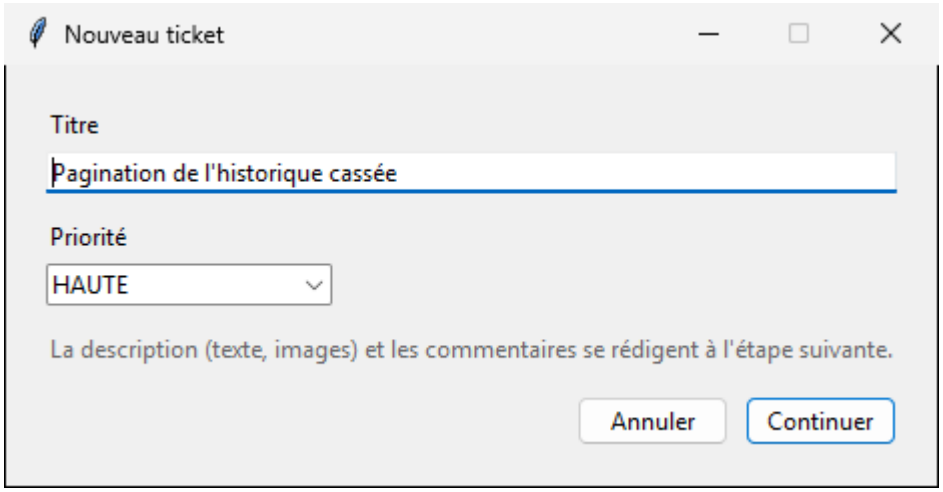


Figure 13. F4 Nouveau ticket : fenêtre réelle.

4.7 F5 Éditeur de description

F5 sert à la création comme à l'édition. Le titre de la fenêtre, l'étiquette « Mode création » ou « Mode édition » et le bouton final, « Créer le ticket » ou « Appliquer », disent dans quel cas on se trouve. Le texte tapé reste dans la zone de texte, avec Ctrl+Z et Ctrl+Y, jusqu'au moment de créer ou d'appliquer. Les images et les commentaires, eux, s'ajoutent aussitôt à la session comme modifications en attente, listées en bas de la fenêtre.

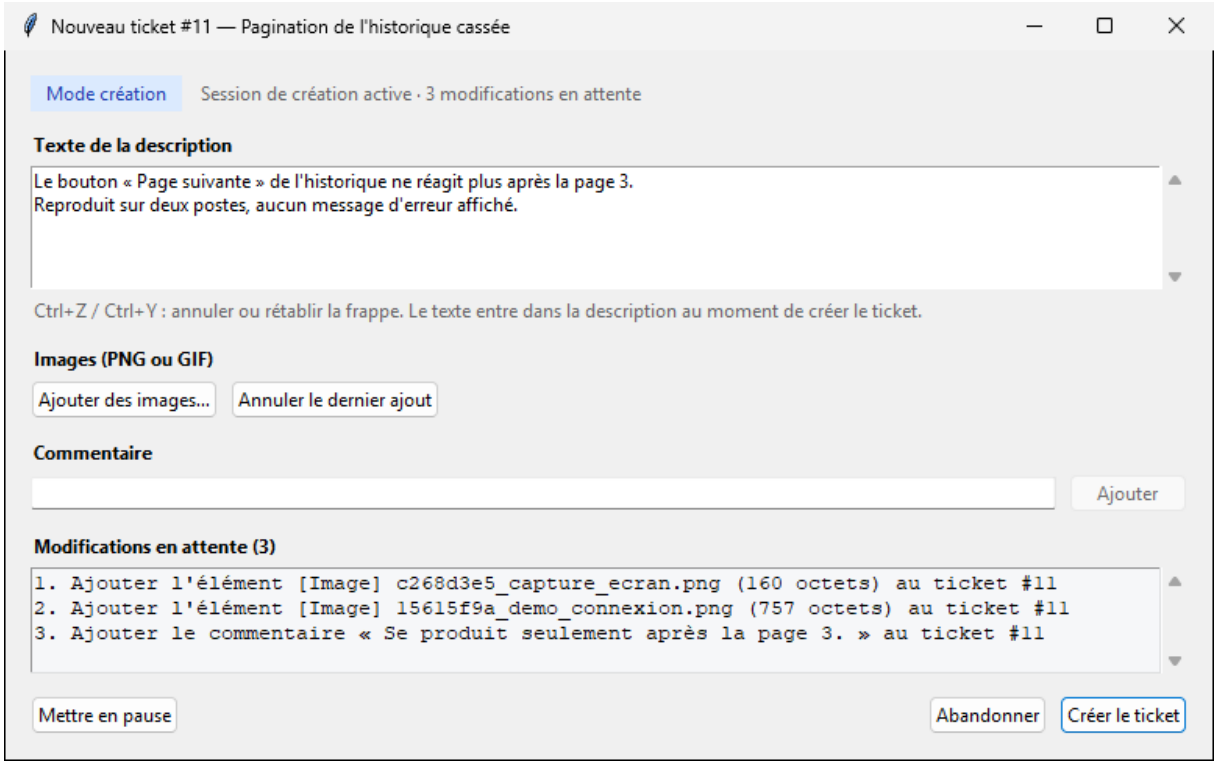


Figure 14. F5 en création : deux images, un commentaire et le texte tapé.

« Annuler le dernier ajout » retire la dernière modification en attente ; si c'est une image, sa copie disparaît de donnees/images/. Plusieurs images s'ajoutent en une seule sélection, et les refus sont réunis dans une seule boîte pendant que les autres images sont ajoutées.

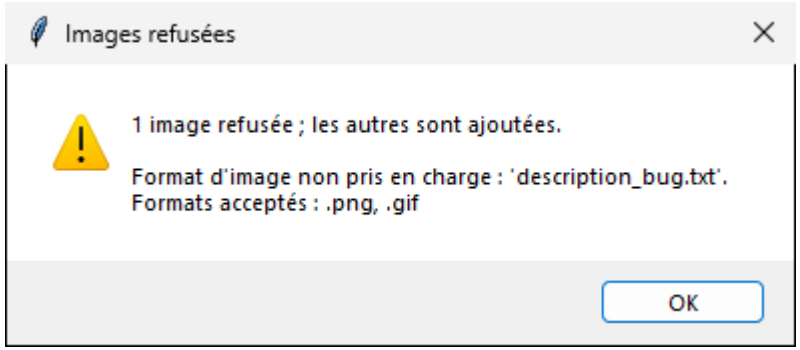


Figure 15. Images refusées : un fichier texte glissé dans la sélection.

« Mettre en pause », ou la croix, ferme F5 sans rien perdre : le texte tapé est gardé par le contrôleur et retrouvé à la reprise depuis F3. « Abandonner » renonce à toute la session, après confirmation si du travail serait perdu.

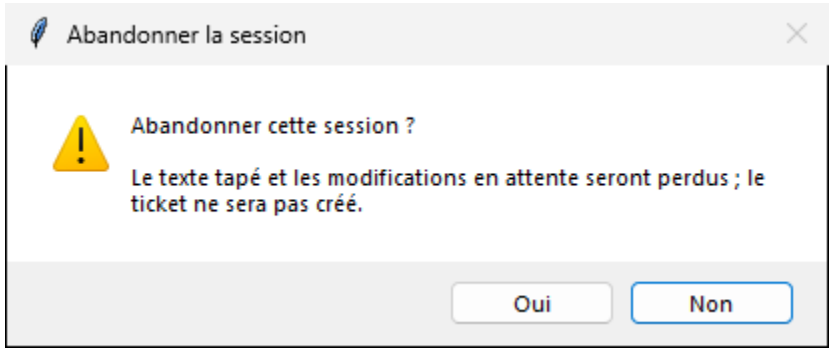


Figure 16. Confirmation d'abandon d'une session de création.

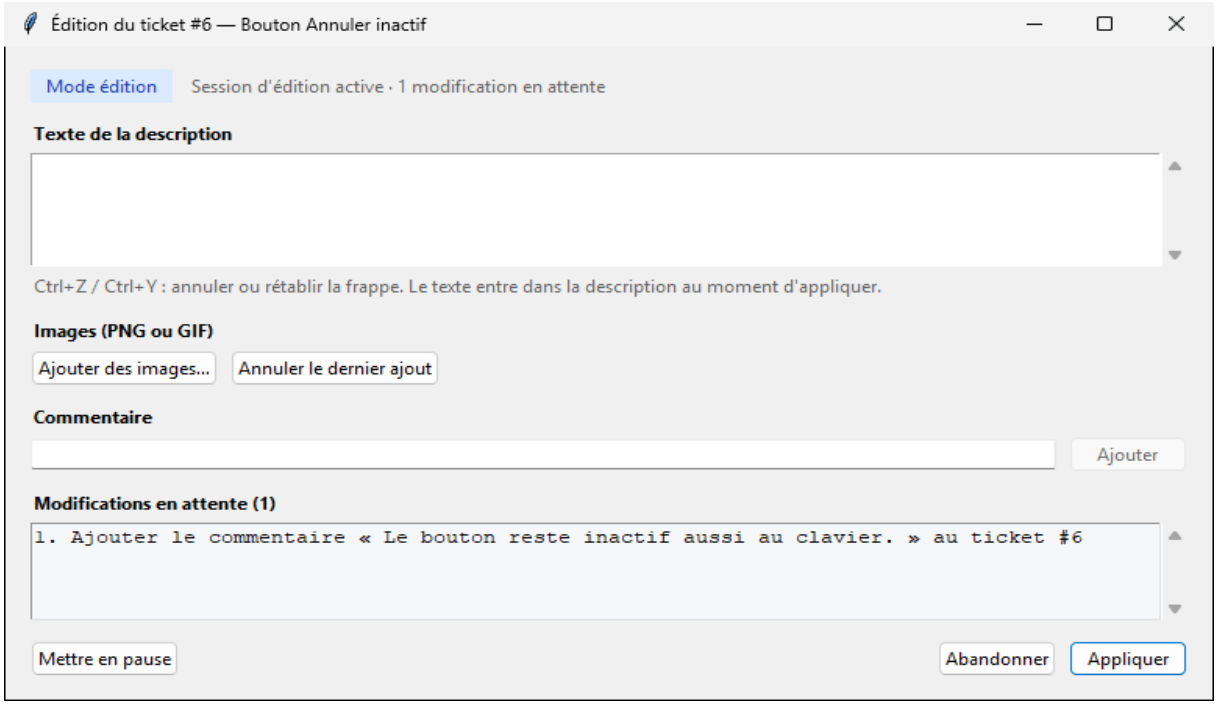


Figure 17. F5 en édition du ticket #6 : bouton « Appliquer ».

4.8 F6 Détail d'un ticket

F6 est commune aux deux rôles et ne fait que lire le ticket : statut, priorité, créateur, assigné et dates, puis la description et les commentaires dans une zone défilante. Le texte et les images s'affichent, une image plus large que 400 pixels étant réduite. « ‹ Précédent » et « Suivant › » parcourent la sélection reçue de F2, ou l'onglet de F3, sans revenir à la liste. Si un autre acteur modifie le ticket affiché, F6 se met à jour ; s'il est retiré du dépôt, un message l'annonce et F6 passe au ticket suivant.

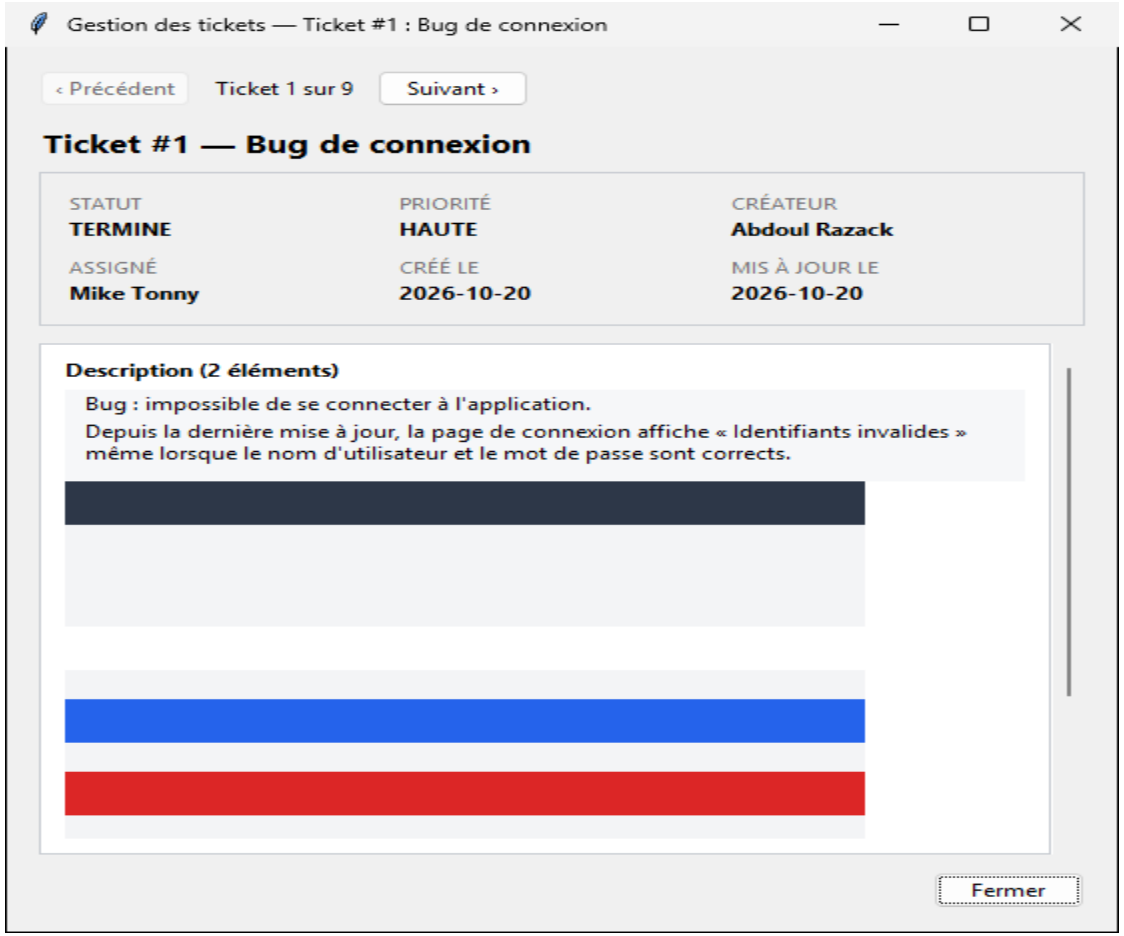


Figure 18. F6 sur le ticket #1 : informations, texte et image de la description.

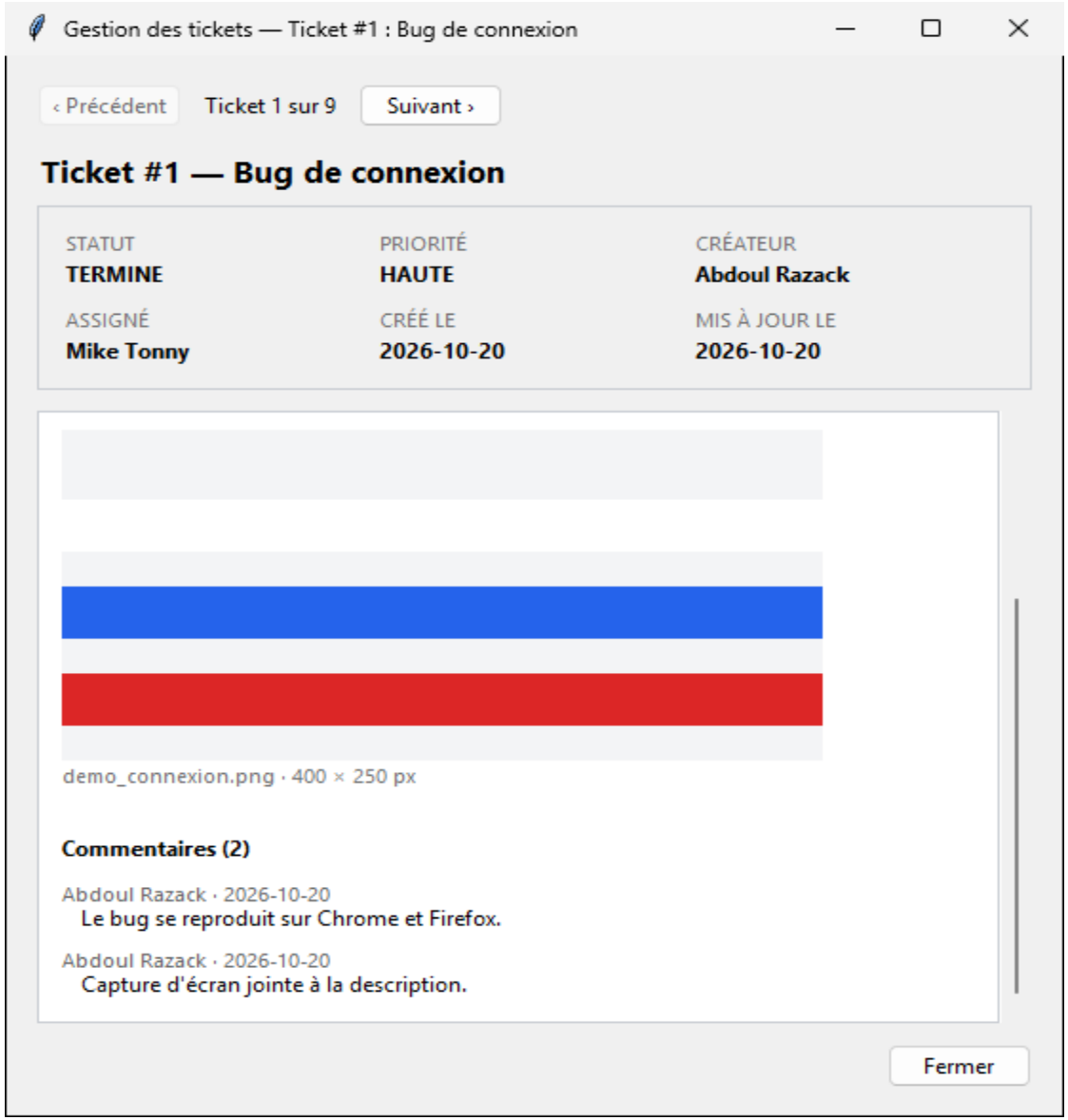


Figure 19. F6 sur le ticket #1, plus bas : les deux commentaires.

5. Tactiques d'utilisabilité

Le tableau 5 relie chaque tactique d'utilisabilité vue en cours à l'endroit où elle apparaît dans l'interface.

Tableau 5. Tactiques d'utilisabilité.

Tactique	Dans l'interface
Annuler	« Annuler » dans F4 et F7 (ou Échap) ferme sans rien appliquer. « Abandonner » dans F5 renonce à toute la session ; les copies d'images sont supprimées et le ticket reste inchangé, ou n'est jamais créé.
Défaire, à deux niveaux	Ctrl+Z et Ctrl+Y défont la frappe dans le texte de F5. « Annuler le dernier ajout » retire la dernière modification en attente de la session.
Pause et reprise	« Mettre en pause », ou la croix de F5, garde la session ouverte. Le brouillon apparaît dans F3, et « Reprendre l'édition » rouvre F5 avec le texte tapé.
Agrégat	Plusieurs images en une sélection dans F5 ; une action de l'Admin appliquée en un seul lot à plusieurs tickets ; F6 qui parcourt toute une sélection.
Modèle du système	Les boutons d'action suivent transitionPossible() : la vue montre ce que le modèle accepterait, sans recopier ses règles.
Prévention des erreurs	Boutons grisés quand le modèle refuserait ; confirmations avec « Non » par défaut ; fermeture définitive avec focus sur « Annuler » ; aucun assigné choisi d'avance dans F7.
Retour d'information	Nombre et liste des modifications en attente dans F5 et F7, nombre de lignes par onglet dans F3, « Ticket n sur N » dans F6, rapport F8 après chaque application.
Cohérence entre fenêtres	Patron Observateur : chaque liste ouverte se met à jour quand un autre acteur modifie le dépôt.

Avec le recul, la tactique la plus utile est aussi la plus discrète : griser un bouton plutôt que laisser l'utilisateur essayer pour lui répondre non. Elle n'est possible que si la vue peut interroger le modèle sans le modifier, ce qui justifiait à lui seul transitionPossible().

6. Tactiques de modificabilité

L'objectif du laboratoire est la modificabilité : pouvoir changer l'interface ou le modèle sans tout reprendre. Le tableau 6 relie les tactiques de modificabilité vues en cours à notre code.

Tableau 6. Tactiques de modificabilité.

Tactique	Dans le code
Augmenter la cohésion	MVC : vues, contrôleurs et modèle dans des paquets distincts. Une vue ne contient aucune règle métier ; changer l'apparence d'une fenêtre ne touche ni au modèle ni à son contrôleur.
Encapsuler	Attributs privés et propriétés en lecture seule. La vue ne lit du modèle que ce qu'il publie, comme transitionPossible() ou modificationsEnAttente.
Utiliser un intermédiaire	Les contrôleurs se placent entre les vues et le modèle. Seul ControleurPrincipal crée des fenêtres : ajouter une fenêtre ne demande de toucher qu'à lui.
Maintenir des interfaces stables	Le contrat est figé dès I1 (constructeurs des vues, signatures des contrôleurs), avec un seul ajout pendant le sprint : nombreImages.
Restreindre les dépendances	Les contrôleurs n'importent pas tkinter : changer de bibliothèque graphique ne toucherait que ihm/vues et ControleurPrincipal.
Abstraire les services communs	FenetreBase regroupe l'abonnement, la fermeture et l'affichage des refus pour les huit fenêtres.
Différer la liaison	L'Observateur notifie les fenêtres sans les connaître ; les fabriques créent descriptions et états à l'exécution ; les données sont chargées depuis un fichier JSON au démarrage.

La preuve la plus concrète : les vues et les contrôleurs ont été écrits en parallèle par deux personnes, sur des fichiers séparés, et le modèle du Lab 2 n'a reçu que des ajouts.

7. Tests et validation

7.1 Tests automatisés

La suite compte 145 tests, lancés par python -m unittest discover -s tests -t . et par GitHub Actions à chaque push et à chaque pull request. Les contrôleurs de l'interface sont testés avec un faux contrôleur principal, qui fournit le dépôt et le stockage des images, et note les rapports qu'on lui demande d'afficher.

Tableau 7. Tests automatisés par fichier.

Fichier	Ce qui est vérifié	Tests
test_controleur_admin.py	Filtre par statut, boutons actifs, préparation et application des lots, rapport	19
test_controleur_user.py	Onglets, brouillons, règles de suppression, de soumission et d'édition	17
test_controleur_edition.py	Agrégat d'images, undo, abandon, pause, application, absence de tkinter	11
test_donnees_demo.py	Chargement de la démonstration : 8 tickets, 5 statuts, aucune vidéo, images présentes	6
test_stockage_images.py	Copie sous un nom unique, formats refusés, suppression limitée au dossier	9
test_observateur.py, test_integration.py	Notifications, relais du dépôt, aucune notification avant appliquer()	20
test_persistance.py	Aller-retour JSON, fichiers illisibles, artefacts introuvables	21
test_sessions.py, test_etats.py, test_description.py, test_systeme_tickets.py	Modèle du Lab 2 et évolutions M1 à M9	41
test_non_regression.py	La sortie de main.py reste celle du Lab 2	1

8. Organisation du travail

Le sprint de l'interface a duré du 17 au 27 octobre. Nous l'avons découpé en tâches à responsable unique : chaque fichier n'appartient qu'à l'un de nous deux, ce qui a évité tout conflit de fusion. Le tableau 8 donne la répartition.

Tableau 8. Répartition des tâches du sprint.

Tâche	Contenu	Qui
R0, I2	Retouches du modèle (SessionEdition.ticket, tests d'intégration) et StockageImages	Julien
I1	Squelette : app.py, FenetreBase, ControleurPrincipal, F1, ébauches, démonstration	Julien
I3	Création et édition : ControleurEdition, F4, F5	Julien
I5, I6	Liste Admin, confirmation du lot et rapport : ControleurAdmin, F2, F7, F8	Abdoul
I7, I8	Mes tickets et détail : ControleurUser, F3, F6	Abdoul
M0	Maquettes Figma de toutes les fenêtres	Julien
U1	Diagramme de classes à jour (paquet ihm, Observateur, persistance)	Abdoul
L1 à L5	Intégration, captures, README, rapport, remise	Les deux

Julien a travaillé sur la branche Zng_Julien, Abdoul sur evolution-ihm-abdoul puis evolution-ihm-abdoul-2, créées depuis main après la fusion du squelette. Chaque groupe de tâches est passé par une pull request relue par l'autre, et fusionnée seulement avec le CI au vert : #3 (R0, I2, I1) et #4 (I3) le 20 octobre, #5 (I5, I6) et #6 (I7, I8) le 21 octobre, puis #7 (U1) le 23 octobre.

9. Difficultés et ce que nous avons appris

Figer le contrat avant le code. Écrire des ébauches qui ne faisaient rien ressemblait d'abord à du temps perdu. C'est pourtant ce qui nous a permis de travailler en parallèle toute la semaine, avec quatre pull requests fusionnées sans un seul conflit.

L'Observateur a un coût de sortie. S'abonner est facile ; le piège est la fenêtre fermée qui reste abonnée. Le désabonnement dans fermer(), avant la destruction, est devenu une règle de FenetreBase plutôt qu'une chose à laquelle chaque vue doit penser.

Une session n'est pas un sujet. F3 doit voir une pause faite dans F5, or une session ne notifie personne. La relecture toutes les 500 ms fonctionne, mais c'est un contournement. Avec plus de temps, nous rendrions les sessions observables, comme les tickets.

Griser plutôt que refuser. Le modèle refusait déjà les opérations interdites par des exceptions, et executer() sait les afficher. Griser les boutons d'avance s'est pourtant révélé plus clair pour l'utilisateur, et plus simple à tester pour nous.

Les fichiers ont une vie hors du modèle. Une image annulée ou abandonnée laissait sa copie dans donnees/images/. Il a fallu une pile de copies alignée sur celle de la session pour savoir quelle copie supprimer, et ne jamais toucher à l'image d'origine.

Quelques limites restent assumées. La connexion ne demande pas de mot de passe. Les sessions ouvertes sont perdues à la fermeture de l'application, ce que la confirmation annonce.

10. Conclusion

L'application couvre les huit fenêtres prévues et le scénario de test manuel en entier. Le modèle du Lab 2 a bien tenu : l'interface n'a demandé que des ajouts (sessions de création, abandon, transitions consultables, Observateur, sauvegarde), et aucune règle métier n'a quitté le modèle pour se retrouver dans une vue. Si nous devions continuer, nous commencerions par rendre les sessions observables, puis par une vraie authentification.

Annexe A. Lancer l'application et les tests

- Interface : python app.py, depuis n'importe quel dossier.
- Démonstration du Lab 2 : python main.py, depuis la racine du dépôt.
- Tests : python -m unittest discover -s tests -t .
- Régénérer les données de démonstration : python outils/generer_demo.py
- Repartir de la démonstration : supprimer donnees/sauvegarde.json.

Annexe B. Structure du code de l'interface

Tableau 9. Fichiers de l'interface.

Fichier ou dossier	Contenu
app.py	Point d'entrée de l'interface
ihm/vues/fenetre_base.py	FenetreBase : abonnement, fermer(), executer()
ihm/vues/vue_*.py	Une vue par fenêtre, F1 à F8
ihm/controleurs/controleur_principal.py	Navigation, chargement, sauvegarde, registre des éditions
ihm/controleurs/controleur_admin.py, controleur_user.py, controleur_edition.py	Contrôleurs sans tkinter
persistance/stockage_images.py	Copie des images dans donnees/images/
outils/generer_demo.py	Génération de donnees/demo.json et de ses images
donnees/	Démonstration versionnée, images, sauvegarde (ignorée par Git)
documents/maquettes/, documents/captures_lab3/	Maquettes Figma et captures du rapport